Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Blood Bike West Coordination Platform

Evan Griffin

B.Sc. (Hons) in Software Development

Final Year Dissertation

Contents

1 Introduction **1**
1.1 Background and Motivation 1
1.2 Project Aim and Objectives 1
1.3 Stakeholder Engagement and Project Context 2
1.4 Development Approach 3
1.5 Scope and Limitations 3
1.6 Structure of the Report 3

2 Literature Review **4**
2.1 Introduction 4
2.2 Web Application Architectures 4
2.3 Progressive Web Applications 5
2.4 Real-Time Tracking Systems 5
2.5 Serverless Cloud Architecture 6
2.6 Security and Authentication 6
2.7 Research Gap 7
2.8 Summary 7

3 System Design and Architecture **8**
3.1 Introduction 8
3.2 System Overview 8
3.3 Architectural Design 9
3.3.1 Frontend Architecture 9
3.3.2 Backend Architecture 10
3.3.3 Cloud Infrastructure 10
3.4 Data Flow 11
3.5 Core System Components 11
3.5.1 Job Dispatch and Delivery Workflow 11
3.5.2 Real-Time Tracking System 11
3.5.3 Authentication and Role-Based Access Control 12
3.5.4 Proof-of-Delivery System 12

3.6 Design Decisions and Trade-Offs . . . 12
3.7 Summary . . . 13

4 Methodology **14**
4.1 Introduction . . . 14
4.2 Development Approach . . . 14
4.3 Stakeholder Engagement . . . 14
4.4 Team Collaboration . . . 15
4.5 Prototyping and Iteration . . . 15
4.6 Testing Strategy . . . 16
4.6.1 Functional Testing . . . 16
4.6.2 Performance Testing . . . 16
4.6.3 Usability Testing . . . 16
4.7 Ethical Considerations . . . 16
4.8 Limitations of Methodology . . . 17
4.9 Summary . . . 17

5 Implementation **18**
5.1 Introduction . . . 18
5.2 System Overview . . . 18
5.3 Frontend Implementation . . . 18
5.3.1 User Interface and Navigation . . . 18
5.3.2 Mapping and Tracking Interface . . . 21
5.4 Backend Implementation . . . 22
5.4.1 API Design . . . 22
5.4.2 Repository Layer . . . 22
5.4.3 Authentication and Security . . . 22
5.5 Core Feature Implementation . . . 23
5.5.1 Job Dispatch System . . . 23
5.5.2 Proof-of-Delivery System . . . 25
5.5.3 Rider Availability System . . . 26
5.5.4 Extended System Features . . . 27
5.6 Infrastructure and Deployment . . . 27
5.7 Changes from Initial Prototype . . . 28
5.8 Summary . . . 28

6 Evaluation **29**
6.1 Introduction . . . 29
6.2 Evaluation Criteria . . . 29
6.3 Functional Evaluation . . . 30
6.3.1 Job Dispatch Workflow . . . 30

6.3.2 Authentication and Role-Based Access . . . 30
6.3.3 Proof-of-Delivery System . . . 30
6.4 Performance Evaluation . . . 31
6.4.1 Performance Metrics . . . 31
6.4.2 Performance Optimisation . . . 31
6.4.3 Root Cause Analysis . . . 32
6.4.4 Optimisation Strategy . . . 32
6.4.5 Discussion of Performance Results . . . 32
6.5 Usability Evaluation . . . 32
6.6 Objective Achievement Analysis . . . 33
6.7 Strengths . . . 33
6.8 Limitations . . . 33
6.9 Critical Discussion . . . 34
6.10 Summary . . . 34

7 Reflection **35**
7.1 Introduction . . . 35
7.2 Stakeholder-Driven Development . . . 35
7.3 Scope Management and Trade-Offs . . . 36
7.4 Technical Challenges . . . 36
7.5 Team Collaboration . . . 36
7.6 Iterative Development and Learning . . . 37
7.7 Personal Reflection . . . 37
7.8 Summary . . . 37

8 Conclusion **39**
8.1 Introduction . . . 39
8.2 Achievement of Objectives . . . 39
8.3 System Evaluation Summary . . . 40
8.4 Impact of Stakeholder Engagement . . . 40
8.5 Limitations of the System . . . 40
8.6 Future Work . . . 41
8.7 Final Reflection . . . 41
8.8 Summary . . . 41

Appendix **42**
Project Resources . . . 42

List of Figures

3.1 System architecture of the Blood Bike West application 9

5.1 Mobile job listing interface showing available delivery tasks 19
5.2 Role selection interface allowing users to switch between assigned system roles 20
5.3 Real-time tracking interface displaying rider location using OpenStreetMap 21
5.4 Job dispatch system interface showing lifecycle progression 23
5.5 Dispatcher interface used to create and manage delivery jobs 24
5.6 Proof-of-delivery system showing signature capture and confirmation 25
5.7 Rider availability dashboard showing active, offline, and assigned riders 26
5.8 Fleet management interface for tracking vehicle readiness and service records 27

List of Tables

6.1 Measured API performance across core endpoints 31
6.2 Performance improvement after optimisation 31
6.3 Evaluation of system objectives 33

Chapter 1

Introduction

1.1 Background and Motivation

Volunteer medical courier services such as Blood Bike West play a critical role in supporting healthcare systems by providing urgent, out-of-hours transportation of blood, medical samples, and essential supplies between hospitals. These services operate primarily through volunteer riders and coordinators, often relying on manual communication methods and ad-hoc coordination tools.

At the outset of this project, Blood Bike West did not possess a dedicated digital platform to manage dispatch operations, rider coordination, or delivery tracking. Existing solutions such as general-purpose messaging applications were either operationally inefficient or unsuitable due to regulatory and compliance considerations, particularly in relation to data protection and privacy requirements. As a result, coordination processes were fragmented, limiting scalability, visibility, and overall efficiency.

This project originated through direct engagement with the Blood Bike community. The development team, consisting of three final-year Computer Science students who are also motorcyclists, became aware of these operational challenges through connections within the motorcycling community. This unique perspective provided both domain insight and motivation to design a system tailored specifically to the needs of such organisations.

1.2 Project Aim and Objectives

The primary aim of this project is to design and implement a scalable, secure, and user-friendly application to support the operational needs of Blood Bike West. The system seeks to streamline dispatch processes, enhance real-time visibility of riders and deliveries, and provide reliable proof-of-delivery mechanisms.

To achieve this aim, the following objectives were defined:

- To develop a role-based web application supporting administrators, dispatchers, and riders
- To implement a job dispatch and delivery lifecycle system for managing medical transport requests
- To provide real-time visibility of rider activity and delivery status
- To enable secure authentication and access control using industry-standard mechanisms
- To implement a digital proof-of-delivery system, including signature capture and automated email receipts
- To design a scalable backend architecture using cloud-based infrastructure
- To ensure the system aligns with operational and regulatory constraints identified by stakeholders

1.3 Stakeholder Engagement and Project Context

A key distinguishing aspect of this project is its stakeholder-driven development approach. The development process began with an in-person requirements elicitation meeting with representatives from Blood Bike West, during which core operational challenges and desired system capabilities were identified.

Following this, an initial prototype was developed and presented to stakeholders via a Microsoft Teams session. This demonstration resulted in several significant design decisions:

- A QR-code-based tracking system, initially implemented during early development, was removed after stakeholders confirmed that dedicated GPS tracking devices were already in use within their fleet.
- A requirement for digital proof-of-delivery was introduced, leading to the implementation of a signature capture interface and automated email receipt system.
- The need for a GDPR-compliant internal messaging system was identified; however, this feature was deferred due to project time constraints and is proposed as future work.

This iterative feedback loop ensured that the system evolved in alignment with real-world operational requirements rather than purely theoretical design assumptions.

1.4 Development Approach

The project was developed using an iterative and collaborative approach, combining agile-inspired practices with continuous stakeholder feedback. The development team conducted weekly planning and review meetings via Discord, during which progress was assessed and tasks were assigned for subsequent development cycles.

Each iteration involved the implementation of new features, followed by testing and validation. Where possible, features were demonstrated to stakeholders to gather feedback, ensuring that development remained aligned with user expectations.

1.5 Scope and Limitations

The scope of the project focused on delivering a functional system capable of supporting core Blood Bike operations, including dispatch, tracking, and delivery confirmation. While the system incorporates a wide range of features, certain limitations were identified:

- A GDPR-compliant messaging system was not implemented due to time constraints
- Large-scale real-world deployment and testing were limited within the project timeframe
- Integration with external hospital systems was not explored

These limitations are discussed further in later chapters of this report.

1.6 Structure of the Report

The remainder of this dissertation is structured as follows:

- Chapter 2 presents a review of relevant literature and technologies
- Chapter 3 details the system design and architecture
- Chapter 4 outlines the development methodology and team contributions
- Chapter 5 describes the implementation of key system features
- Chapter 6 evaluates the system through testing and analysis
- Chapter 7 concludes the report and discusses future work

Chapter 2

Literature Review

2.1 Introduction

This chapter reviews the existing research and technologies relevant to the development of the Blood Bike West application. It focuses on key areas including web application architectures, Progressive Web Applications (PWAs), real-time tracking systems, serverless cloud infrastructure, and authentication mechanisms. The aim of this chapter is to establish a theoretical foundation for the system design decisions made in this project, while also identifying limitations in existing approaches and highlighting a gap in current research.

2.2 Web Application Architectures

Modern web applications are typically built using a client-server architecture, where a frontend user interface communicates with backend services via APIs. This separation of concerns improves scalability, maintainability, and flexibility, allowing systems to evolve independently across different layers [1].

REST (Representational State Transfer) is one of the most widely adopted architectural styles for web services. Fielding [1] defines REST as a stateless architecture that enables distributed systems to scale efficiently through uniform interfaces. RESTful APIs are particularly well-suited to cloud-based environments due to their simplicity and compatibility with HTTP-based communication.

However, REST-based systems also introduce limitations. Since REST is stateless, each request must contain all necessary information, which can increase overhead in applications requiring frequent updates. In systems such as logistics platforms or tracking applications, this can lead to increased latency and inefficiency.

Alternative approaches such as GraphQL or event-driven architectures aim to address these limitations by reducing over-fetching or enabling asynchronous

communication. However, these approaches introduce additional complexity and may not be necessary for systems operating at smaller scales. As a result, REST remains a practical and widely supported choice for many real-world applications.

2.3 Progressive Web Applications

Progressive Web Applications (PWAs) represent a modern approach to web development, combining the accessibility of traditional web applications with features typically associated with native mobile apps [2].

PWAs offer several advantages, including cross-platform compatibility, reduced development costs, and simplified deployment. Unlike native applications, which require separate implementations for different operating systems, PWAs can be accessed through a browser and installed directly onto a device. This makes them particularly suitable for organisations with limited development resources.

In addition, PWAs support features such as offline functionality, push notifications, and background synchronisation through the use of service workers. These capabilities enhance user experience, particularly in mobile-first environments.

However, PWAs also have limitations. Compared to native applications, they provide restricted access to certain device-level features and may suffer from reduced performance in scenarios requiring continuous background processing. This is particularly relevant for applications involving real-time tracking or frequent data updates.

As a result, while PWAs provide a highly accessible and cost-effective solution, developers must carefully evaluate whether their limitations impact the specific requirements of the system being developed.

2.4 Real-Time Tracking Systems

Real-time tracking is a fundamental requirement in logistics and delivery systems. Traditional tracking systems rely on GPS technology to determine location, combined with backend services to process and display this data [3].

Modern web-based tracking systems typically use either WebSockets or HTTP polling to deliver updates to users. WebSockets enable persistent, bidirectional communication between client and server, allowing for low-latency updates and real-time interaction.

However, research by Pimentel and Nickerson [4] indicates that WebSocket implementations introduce additional complexity, including connection management and increased resource consumption. These factors can make them more difficult to implement and maintain, particularly in smaller-scale systems.

In contrast, HTTP polling involves periodically sending requests to the server to retrieve updated information. While this approach introduces latency and increased network traffic, it is significantly simpler to implement and more compatible with existing infrastructure.

For systems with moderate update frequency, such as volunteer-based delivery networks, polling can provide an acceptable balance between performance and simplicity. This trade-off is particularly relevant when development resources and system complexity must be carefully managed.

2.5 Serverless Cloud Architecture

Serverless computing has emerged as a scalable and cost-efficient model for backend development. In a serverless architecture, applications are composed of functions that are executed in response to events, without requiring developers to manage underlying infrastructure [5].

Platforms such as AWS Lambda provide automatic scaling, allowing systems to handle varying workloads without manual intervention. This is particularly advantageous for applications with unpredictable usage patterns, such as volunteer-based services where demand may fluctuate.

Serverless architectures also reduce operational overhead, as tasks such as provisioning, scaling, and maintenance are handled by the cloud provider. This enables developers to focus on application logic rather than infrastructure management.

However, serverless computing introduces several challenges. One of the most notable is cold start latency, where functions experience delays when invoked after periods of inactivity. This can impact performance in latency-sensitive applications.

Additionally, serverless systems introduce dependency on cloud providers, raising concerns related to vendor lock-in and portability. These trade-offs must be considered when designing systems that rely heavily on cloud infrastructure.

2.6 Security and Authentication

Security is a critical consideration in web-based systems, particularly those handling operational or sensitive data. JSON Web Tokens (JWT) are widely used as a stateless authentication mechanism, enabling secure communication between clients and servers [6].

JWT-based authentication involves issuing a token upon successful login, which is then included in subsequent requests. This approach eliminates the need for server-side session storage, improving scalability and simplifying backend design.

Cloud-based authentication services, such as AWS Cognito, provide additional functionality by handling user management, authentication flows, and access control. These services reduce development complexity and improve security by leveraging established frameworks.

However, reliance on external authentication providers introduces dependency on third-party systems. This may limit flexibility and control over authentication workflows, particularly in applications requiring custom security policies.

2.7 Research Gap

While extensive research exists on logistics systems, real-time tracking, and cloud-based architectures, the majority of studies focus on large-scale commercial applications. These systems are typically designed for high-volume operations with significant infrastructure and financial resources.

In contrast, volunteer-based organisations such as Blood Bike West operate under fundamentally different conditions. These include reliance on volunteer availability, limited resources, and the need for rapid response in time-critical situations.

There is limited research addressing the design of systems specifically tailored to volunteer-based medical courier services. Existing solutions do not fully account for the unique operational constraints and requirements of such organisations.

This project aims to address this gap by developing a system that balances functionality, simplicity, and scalability, specifically designed to support a volunteer-driven logistics network.

2.8 Summary

This chapter has reviewed the key technologies and research areas relevant to the project, including web architectures, PWAs, real-time tracking systems, serverless computing, and authentication mechanisms.

It has highlighted the strengths and limitations of each approach, while also identifying a gap in existing research. These insights directly informed the design and implementation decisions presented in the following chapters.

Chapter 3

System Design and Architecture

3.1 Introduction

This chapter presents the design and architecture of the Blood Bike West application. It outlines the overall system structure, key components, data flow, and design decisions made during development. The system was designed to be scalable, modular, and aligned with the operational requirements identified through stakeholder engagement.

3.2 System Overview

The system follows a client-server architecture consisting of a Progressive Web Application (PWA) frontend, a backend API implemented in Go, and a cloud-based infrastructure hosted on Amazon Web Services (AWS).

The frontend provides an interactive user interface for riders, dispatchers, and administrators, while the backend exposes RESTful APIs responsible for handling business logic, authentication, and data persistence. Cloud services are used to ensure scalability, reliability, and secure access control.

As shown in Figure 3.1, the system is structured in a layered format, with clear separation between presentation, application logic, and data storage.

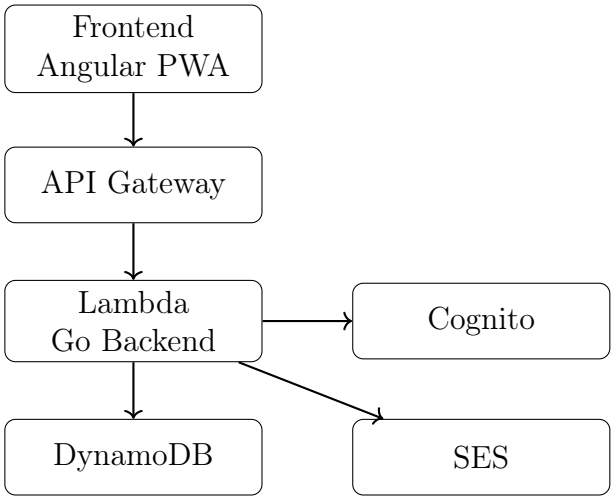


Figure 3.1: System architecture of the Blood Bike West application

At a high level, the system consists of:

- **Frontend:** Angular-based Progressive Web Application
- **Backend:** Go-based REST API
- **Infrastructure:** AWS Lambda, API Gateway, DynamoDB, and Cognito
- **External Services:** Email delivery via AWS SES and mapping via OpenStreetMap

3.3 Architectural Design

The system adopts a layered architecture, separating concerns between presentation, application logic, and data storage. This approach improves maintainability, scalability, and the ability to evolve individual components independently.

3.3.1 Frontend Architecture

The frontend is implemented using Angular and structured as a component-based single-page application. It uses routing to provide role-specific views for different types of users, including administrators, dispatchers, and riders.

Key features of the frontend architecture include:

- Component-based UI design for modularity and reusability
- Service-based data access using Angular's HttpClient

- Reactive state management using signals
- Integration with mapping libraries for real-time location display

The use of a Progressive Web Application enables features such as installation on mobile devices and push notifications. This was particularly important given the mobile-centric nature of rider workflows.

3.3.2 Backend Architecture

The backend is implemented in Go using a modular structure, organised into domain-specific packages. It follows a RESTful design, exposing endpoints under the `/api` path.

Key architectural elements include:

- HTTP routing layer using Go's `net/http` package
- Middleware for authentication and role-based access control
- Domain-driven package structure (e.g., jobs, tracking, fleet, events)
- Repository abstraction layer for database interaction

The backend is designed to be stateless, enabling horizontal scaling through serverless deployment. This aligns with the requirement for scalability while minimising operational overhead.

3.3.3 Cloud Infrastructure

The application is deployed using AWS serverless technologies. The infrastructure is defined using AWS Cloud Development Kit (CDK), allowing infrastructure to be managed as code.

Core services include:

- **API Gateway:** Entry point for all HTTP requests
- **AWS Lambda:** Hosts backend API logic
- **DynamoDB:** NoSQL database for application data
- **Cognito:** User authentication and role management
- **SES:** Email service for delivery receipts

This architecture provides scalability and fault tolerance while reducing the need for manual infrastructure management. However, it also introduces trade-offs, such as cold start latency and dependency on cloud services.

3.4 Data Flow

The system follows a structured request-response lifecycle:

1. A user performs an action via the frontend (e.g., creating a job)
2. The request is sent to the backend via API Gateway
3. Authentication is verified using JWT tokens issued by Cognito
4. The backend processes the request and interacts with the database
5. A response is returned to the frontend and rendered to the user

This architecture ensures secure and consistent communication between system components while maintaining separation of concerns.

3.5 Core System Components

3.5.1 Job Dispatch and Delivery Workflow

The job dispatch system is central to the application. Dispatchers can create delivery jobs, which are then visible to riders. Riders can accept jobs and progress them through a defined lifecycle:

- **Open:** Job created and awaiting acceptance
- **Accepted:** Rider assigned to job
- **Picked-Up:** Item collected by rider
- **Delivered:** Delivery completed

This structured workflow ensures clear tracking of delivery status and accountability.

3.5.2 Real-Time Tracking System

The system provides real-time tracking of riders using location updates. These updates are processed by the backend and made available to the frontend for visualisation on a map interface.

Location data is stored in an in-memory structure for fast access and is periodically updated. While this approach provides good performance and simplicity, it does not offer long-term persistence of tracking data.

3.5.3 Authentication and Role-Based Access Control

Authentication is implemented using AWS Cognito and JSON Web Tokens (JWT). Users must authenticate before accessing protected endpoints.

Role-based access control ensures that users only have access to functionality relevant to their role, improving both security and usability.

3.5.4 Proof-of-Delivery System

A key feature introduced following stakeholder feedback is the proof-of-delivery system. This allows recipients to digitally sign for deliveries, with the signature captured via the frontend interface.

Once a delivery is completed:

- A signature is recorded
- A receipt is generated by the backend
- The receipt is sent via email using AWS SES

This provides verifiable confirmation of completed deliveries and improves accountability within the system.

3.6 Design Decisions and Trade-Offs

Several important design decisions were made during development:

- **Removal of QR Tracking:** Although a QR-based system was implemented, stakeholder feedback indicated that existing GPS tracking devices made this feature redundant. Its removal simplified the system and aligned it more closely with real-world workflows.
- **Use of Serverless Architecture:** AWS Lambda was chosen to reduce infrastructure management and allow automatic scaling. However, this introduces potential latency due to cold starts.
- **Polling over WebSockets:** Polling was selected due to its simplicity and reliability for the expected scale. While WebSockets would provide lower latency, they introduce additional complexity and overhead.
- **Deferred Messaging Feature:** A GDPR-compliant messaging system was identified but deferred due to time constraints, reflecting the need to prioritise core functionality.

These decisions demonstrate a balance between technical capability, stakeholder requirements, and practical constraints.

3.7 Summary

This chapter has outlined the architecture and design of the Blood Bike West application. The system leverages modern web technologies and cloud infrastructure to deliver a scalable and user-focused solution.

Chapter 4

Methodology

4.1 Introduction

This chapter outlines the methodology used in the development of the Blood Bike West application. It describes the development approach, stakeholder engagement process, team collaboration, and testing strategies used throughout the project. The aim is to demonstrate how the system was developed in a structured and systematic manner.

4.2 Development Approach

An iterative and incremental development approach was adopted for this project, influenced by Agile methodologies. Agile development emphasises flexibility, continuous feedback, and incremental delivery of functionality.

This approach was particularly suitable for this project due to the involvement of an external stakeholder (Blood Bike West), whose requirements evolved over time. Rather than attempting to define all system requirements upfront, development was carried out in cycles, allowing features to be implemented, reviewed, and refined based on feedback.

Each iteration focused on delivering a small set of features, which were then tested and evaluated before moving on to the next stage. This ensured that the system remained aligned with stakeholder expectations throughout development.

4.3 Stakeholder Engagement

A key aspect of the methodology was direct engagement with Blood Bike West. Initial requirements were gathered through an in-person meeting, where the or-

ganisation outlined their operational challenges and desired system features.

Following this, a prototype of the application was developed and presented during an online meeting. Feedback from this session led to several important changes, including the removal of the QR tracking system and the introduction of proof-of-delivery functionality.

Regular communication was maintained throughout the project to ensure that the system continued to meet stakeholder needs. This iterative feedback process played a crucial role in shaping the final system.

4.4 Team Collaboration

The project was developed collaboratively by a team of three members. Responsibilities were distributed across different areas of the system, including frontend development, backend implementation, and infrastructure configuration.

Weekly meetings were held using Discord to review progress, discuss challenges, and plan upcoming work. This ensured that all team members remained aligned and that dependencies between components were managed effectively.

Version control was managed using GitHub, allowing changes to be tracked and contributions to be coordinated. This also provided a clear record of development progress over time.

While the project was developed collaboratively, my primary contributions focused on backend API development using Go, cloud infrastructure design using AWS CDK, authentication and security implementation using Cognito and JWT, and system performance optimisation.

4.5 Prototyping and Iteration

The development process began with the creation of a basic prototype, which focused on core functionality such as job creation and user authentication. This prototype served as a foundation for further development and allowed early validation of design decisions.

Based on stakeholder feedback, additional features were introduced incrementally. These included real-time tracking, rider availability management, and proof-of-delivery.

This iterative approach allowed the system to evolve over time, reducing the risk of developing features that did not align with user requirements.

4.6 Testing Strategy

A combination of testing approaches was used to evaluate the system, including functional testing, performance testing, and usability testing.

4.6.1 Functional Testing

Functional testing was conducted to ensure that all system features operated as expected. This included testing user authentication, job creation, job lifecycle transitions, and proof-of-delivery functionality.

Edge cases were also considered, such as invalid input data and unauthorised access attempts. This helped ensure that the system behaved correctly under a range of conditions.

4.6.2 Performance Testing

Performance testing was conducted using a custom simulation tool developed as part of the project. This tool simulated multiple users interacting with the system simultaneously, allowing response times and system behaviour to be evaluated under load.

The results of this testing are presented in the Evaluation chapter, where system performance is analysed in detail.

4.6.3 Usability Testing

Basic usability testing was carried out with a small group of users. Participants were asked to complete common tasks, such as accepting a job and updating availability.

Feedback from these tests was used to improve the user interface and ensure that the system was intuitive and easy to use.

4.7 Ethical Considerations

As the system is designed for a real-world organisation, ethical considerations were taken into account during development. These included ensuring secure handling of user data and compliance with relevant data protection regulations.

The use of AWS Cognito for authentication and secure communication protocols helped ensure that sensitive information was protected.

4.8 Limitations of Methodology

While the methodology adopted was effective for this project, several limitations were identified. The scale of testing was limited, particularly in terms of the number of users involved in usability testing.

Additionally, due to time constraints, some features such as a GDPR-compliant messaging system were not implemented. These limitations highlight areas for future improvement.

4.9 Summary

This chapter has described the methodology used in developing the Blood Bike West application, including the development approach, stakeholder engagement, and testing strategies.

The iterative and stakeholder-driven approach ensured that the system remained aligned with real-world requirements, while the testing process provided a basis for evaluating system performance and usability in the following chapter.

Chapter 5

Implementation

5.1 Introduction

This chapter describes the implementation of the Blood Bike West application, detailing how the system design outlined in the previous chapter was realised in practice. It focuses on the development of key system features, including job dispatch, authentication, real-time tracking, and proof-of-delivery.

5.2 System Overview

The system was implemented as a full-stack web application consisting of a Progressive Web Application (PWA) frontend built using Angular, and a backend API developed in Go. The application is deployed using AWS serverless infrastructure, enabling scalability and high availability.

The implementation follows a modular architecture, with separate components responsible for authentication, job management, tracking, and user interaction.

5.3 Frontend Implementation

5.3.1 User Interface and Navigation

The user interface was designed with a mobile-first approach, reflecting the primary use case of riders accessing the system on mobile devices. Navigation is handled through Angular routing, enabling different views based on user roles.

Dynamic UI elements were implemented to reflect system state in real time, such as job status updates and rider availability.

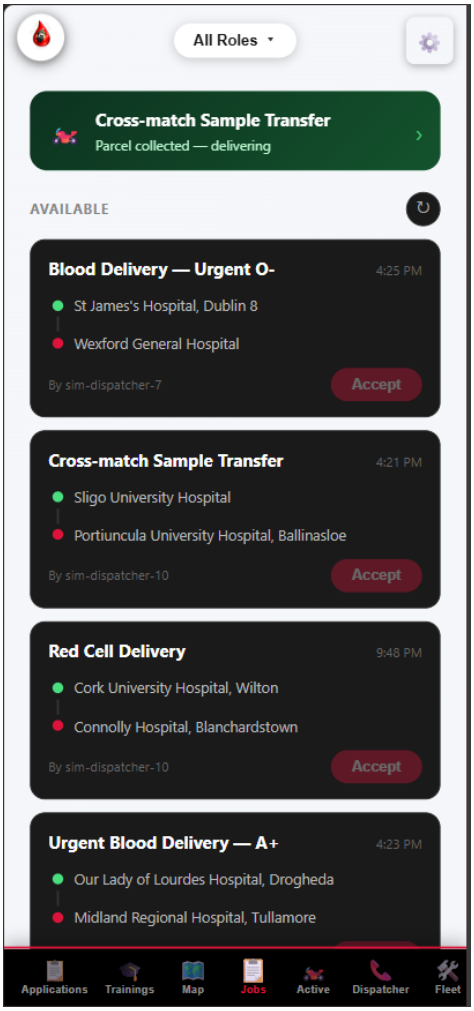


Figure 5.1: Mobile job listing interface showing available delivery tasks

This interface demonstrates how the system presents available jobs in a clear and accessible format, supporting efficient task selection by riders.

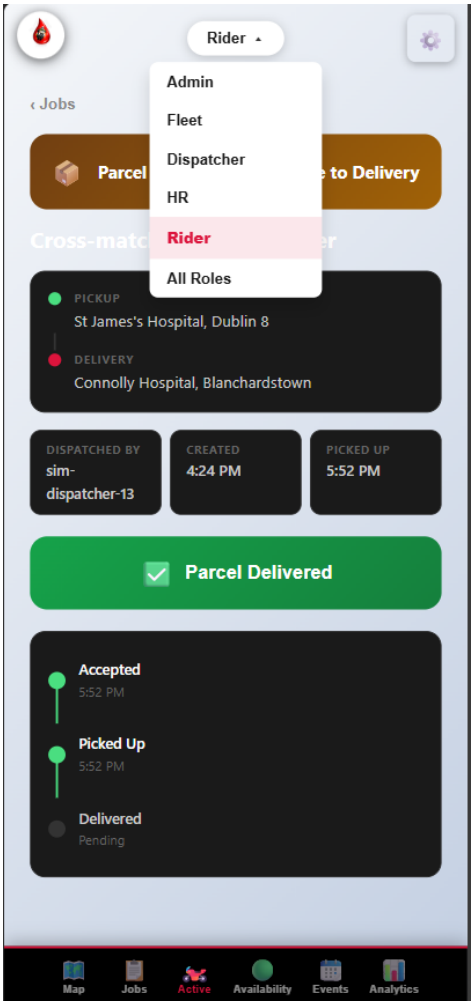


Figure 5.2: Role selection interface allowing users to switch between assigned system roles

The system supports a multi-role user model, where users can be assigned one or more roles at the point of account creation by an administrator. Unlike traditional systems where a user is restricted to a single role, this approach allows individuals to operate across multiple functional areas of the application.

For example, a user assigned both *rider* and *dispatcher* roles can switch between these roles within the interface, gaining access to the relevant functionality for each context. In contrast, users assigned a single role are restricted to that role only.

This design was chosen to reflect the operational structure of volunteer-based organisations, where individuals often take on multiple responsibilities. By enabling dynamic role switching, the system reduces the need for multiple user accounts while maintaining clear separation of permissions through role-based access

control.

5.3.2 Mapping and Tracking Interface

The application integrates a mapping interface using Leaflet and OpenStreetMap.

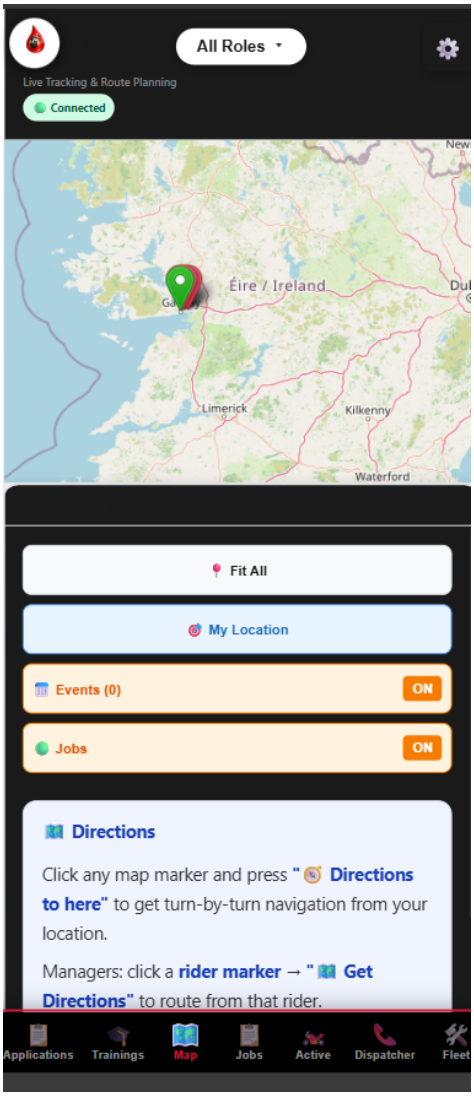


Figure 5.3: Real-time tracking interface displaying rider location using OpenStreetMap

This interface highlights the system's ability to provide live operational awareness in time-critical scenarios.

5.4 Backend Implementation

5.4.1 API Design

The API follows REST principles.

5.4.2 Repository Layer

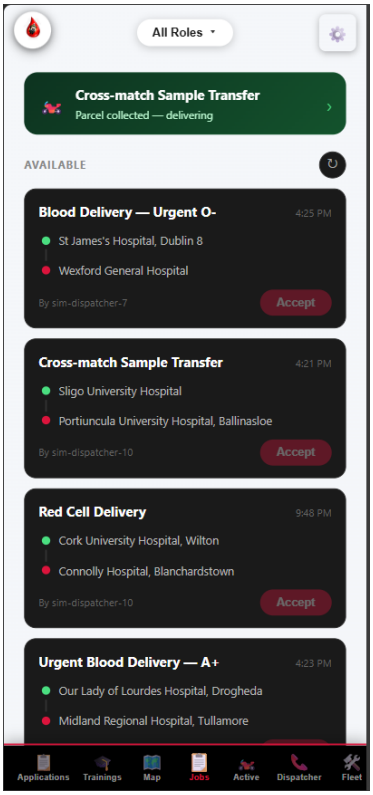
A repository pattern was used to abstract database interactions.

5.4.3 Authentication and Security

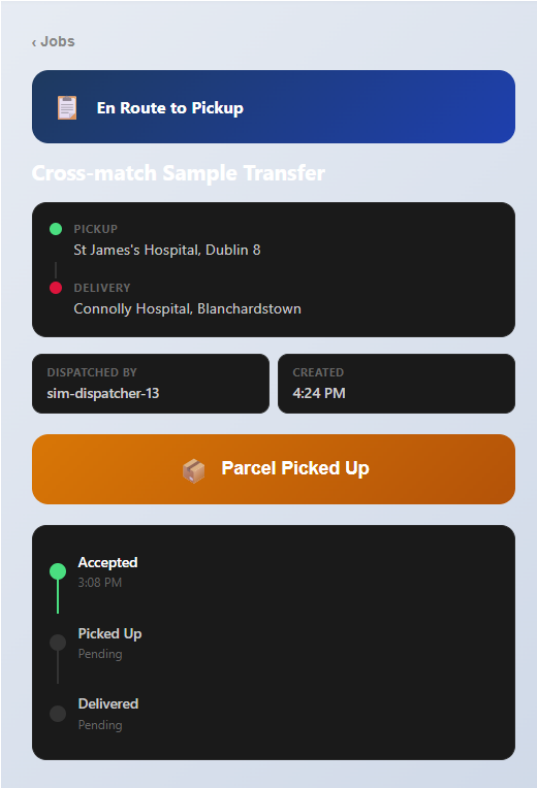
Authentication is implemented using AWS Cognito and JSON Web Tokens (JWT).

5.5 Core Feature Implementation

5.5.1 Job Dispatch System



(a) Available jobs



(b) Active job lifecycle

Figure 5.4: Job dispatch system interface showing lifecycle progression

These interfaces demonstrate how jobs progress through the system lifecycle.

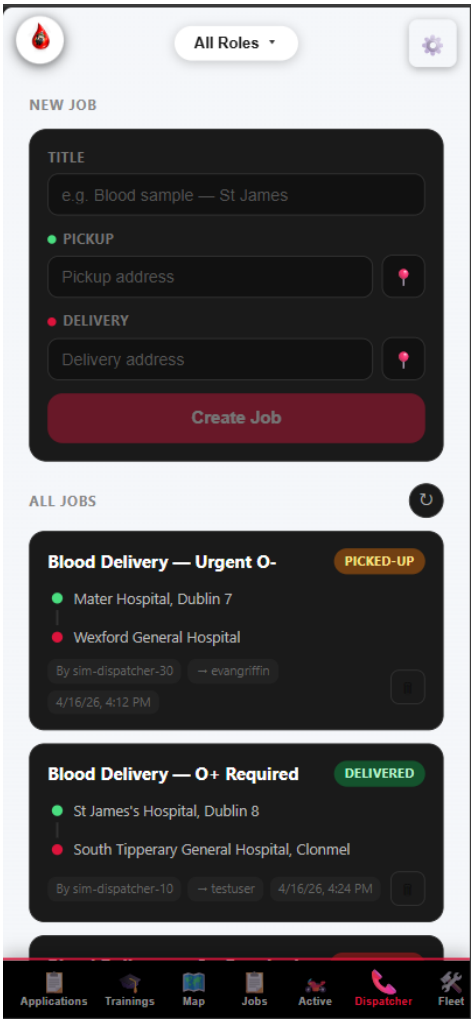
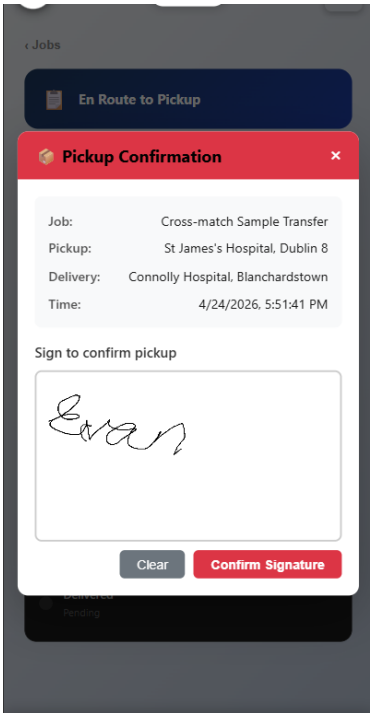
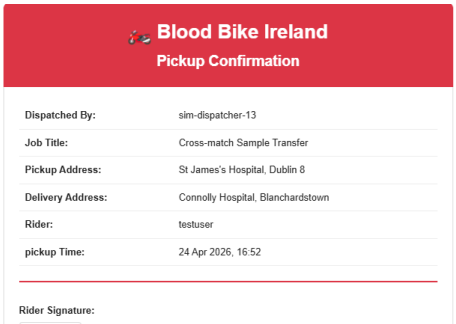


Figure 5.5: Dispatcher interface used to create and manage delivery jobs

5.5.2 Proof-of-Delivery System



(a) **Signature capture**



(b) **Email confirmation**

Figure 5.6: Proof-of-delivery system showing signature capture and confirmation

This feature provides verifiable confirmation of deliveries.

5.5.3 Rider Availability System

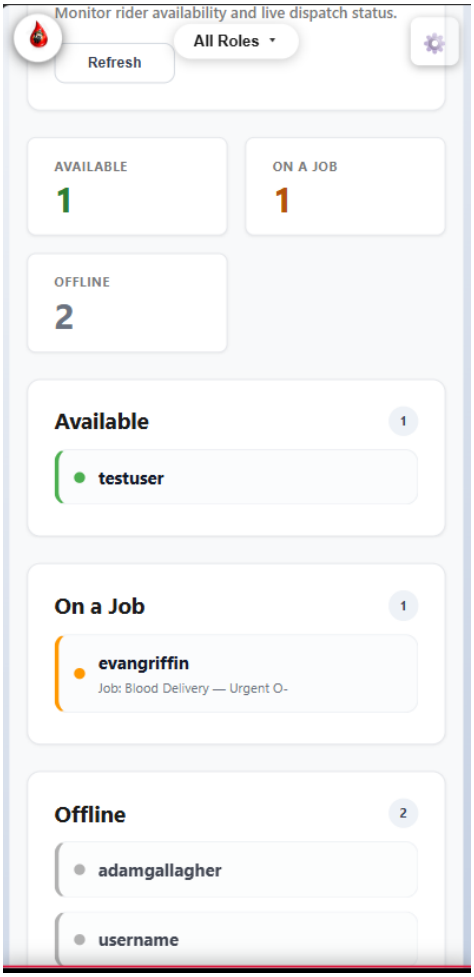


Figure 5.7: Rider availability dashboard showing active, offline, and assigned riders

This improves operational visibility for dispatchers.

5.5.4 Extended System Features

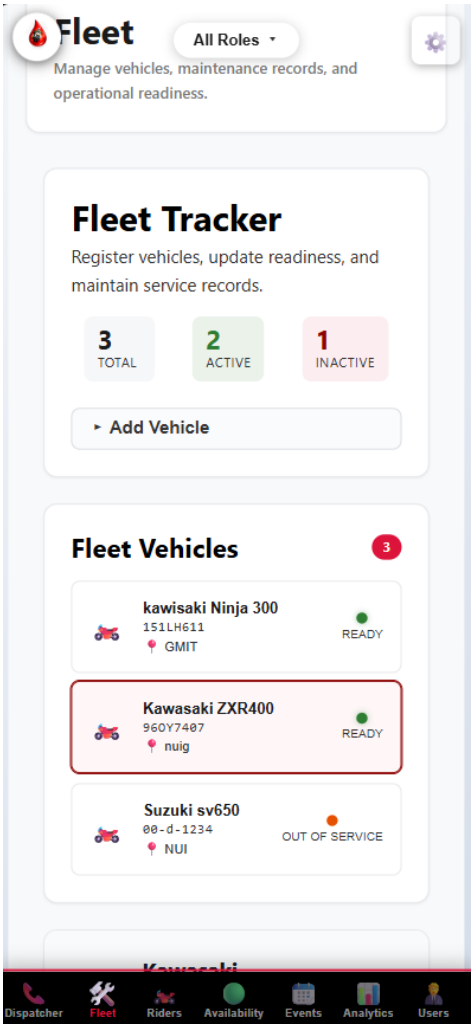


Figure 5.8: Fleet management interface for tracking vehicle readiness and service records

This demonstrates the extensibility of the system beyond core dispatch functionality.

5.6 Infrastructure and Deployment

The system is deployed using AWS serverless infrastructure.

5.7 Changes from Initial Prototype

Several changes were made based on stakeholder feedback.

5.8 Summary

This chapter has described the implementation of the Blood Bike West application.

Chapter 6

Evaluation

6.1 Introduction

This chapter evaluates the Blood Bike West application in terms of functionality, performance, usability, and alignment with the project objectives defined earlier. The aim is to assess how effectively the system meets stakeholder requirements and to identify both strengths and limitations of the implementation.

6.2 Evaluation Criteria

The system was evaluated against the following key objectives:

- Implementation of a role-based system
- Functionality of the job dispatch workflow
- Real-time visibility of riders and job status
- Secure authentication and access control
- Proof-of-delivery with email confirmation
- System performance under concurrent usage

Each of these criteria is assessed in the following sections.

6.3 Functional Evaluation

6.3.1 Job Dispatch Workflow

The job dispatch system was tested across all lifecycle stages, including job creation, assignment, progression, and completion.

Testing confirmed that:

- Dispatchers could successfully create and assign jobs
- Riders could view and accept available jobs
- Jobs progressed correctly through defined states

State validation logic ensured that invalid transitions were prevented, improving system reliability and consistency.

6.3.2 Authentication and Role-Based Access

Authentication was evaluated using both valid and invalid credentials. Role-based access control was verified by ensuring that users could only access functionality relevant to their assigned role.

For example:

- Riders were restricted from administrative functionality
- Administrators retained full system access

This confirms that the authentication system effectively enforces access control.

6.3.3 Proof-of-Delivery System

The proof-of-delivery system was tested by completing delivery workflows and verifying that:

- Signatures were successfully captured
- Delivery status was updated correctly
- Email receipts were generated and delivered

An issue was encountered where emails were initially marked as spam. This was resolved through the implementation of domain verification and authentication mechanisms such as SPF and DKIM.

6.4 Performance Evaluation

6.4.1 Performance Metrics

Performance testing was conducted using a custom simulation tool developed as part of this project. The tool simulated multiple users interacting with the system simultaneously.

Endpoint	Avg Response Time (ms)	Samples
GET /api/jobs	2.6	68
POST /api/auth/signin	4.7	36
POST /api/jobs	1.4	101
PUT /api/jobs/id	52.2	195
PUT /api/riders/availability/me	68.1	71

Table 6.1: Measured API performance across core endpoints

The results demonstrate that the system is capable of handling requests efficiently, with most endpoints responding within a few milliseconds.

6.4.2 Performance Optimisation

Initial testing revealed significant performance issues.

Metric	Before Optimisation	After Optimisation
Average Response Time	820.8 ms	2.6 ms

Table 6.2: Performance improvement after optimisation

The observed performance improvement was primarily the result of correcting an environment configuration issue rather than algorithmic optimisation. During initial testing, the application was unintentionally communicating with a remote DynamoDB instance despite running in local development mode. This introduced significant network latency.

By switching to an in-memory data store for local testing, the system eliminated unnecessary network overhead, resulting in a substantial reduction in response time.

This highlights the importance of correct environment configuration in distributed systems, where performance bottlenecks may arise from infrastructure rather than application logic.

6.4.3 Root Cause Analysis

The performance issue was traced to incorrect environment configuration. Although the system was running in local development mode, it continued to interact with a remote DynamoDB instance.

This resulted in:

- High network latency
- Excessive API calls
- Unintended writes to production data

6.4.4 Optimisation Strategy

The issue was resolved through:

- Enforcing in-memory storage in development mode
- Introducing cleanup mechanisms for test data
- Adding a Global Secondary Index (GSI) for efficient querying

These changes significantly improved performance and reduced unnecessary system overhead.

6.4.5 Discussion of Performance Results

The results demonstrate that the system performs efficiently under moderate load conditions. The use of serverless architecture enables automatic scaling, allowing the system to handle varying workloads.

However, certain endpoints exhibited higher response times due to additional processing requirements. While acceptable for current usage, further optimisation may be required for larger-scale deployment.

6.5 Usability Evaluation

Usability testing was conducted with a small group of users. Participants were asked to perform common tasks, including accepting jobs and updating availability.

The results indicated that:

- The system was intuitive and easy to navigate
- Users were able to complete tasks without instruction

- The mobile-friendly design improved accessibility

Minor feedback included:

- Desire for faster tracking updates
- Suggestions for improved visual indicators

6.6 Objective Achievement Analysis

Objective	Achieved	Evidence
Job Dispatch System	Yes	Fully functional lifecycle implementation
Real-Time Tracking	Partial	Implemented via polling with minor delay
Authentication	Yes	Secure JWT-based access control
Proof-of-Delivery	Yes	Signature capture and email receipts
Scalability	Partial	Tested under moderate load only

Table 6.3: Evaluation of system objectives

6.7 Strengths

- Strong alignment with stakeholder requirements
- Scalable serverless architecture
- Effective real-time tracking and dispatch system
- Robust authentication and access control

6.8 Limitations

- Messaging system not implemented
- Limited large-scale performance testing
- Polling introduces latency compared to WebSockets

6.9 Critical Discussion

The evaluation demonstrates that the system successfully meets its core objectives and provides practical value. The iterative approach allowed for continuous improvement based on stakeholder feedback.

The identification and resolution of performance issues highlights the importance of testing and optimisation. While the system performs well within expected usage, further work is required to ensure scalability for larger deployments.

6.10 Summary

This chapter has evaluated the system across multiple criteria, demonstrating that it meets its primary objectives while identifying areas for improvement. The results confirm that the system provides a strong foundation for future development.

Chapter 7

Reflection

7.1 Introduction

This chapter provides a critical reflection on the development of the Blood Bike West application. It evaluates the decisions made throughout the project, highlights key challenges encountered, and discusses lessons learned from both technical and collaborative perspectives.

7.2 Stakeholder-Driven Development

One of the most significant strengths of this project was the direct engagement with Blood Bike West as an external stakeholder. Unlike purely academic projects, the system was developed to address real operational challenges, which provided valuable insight into practical software engineering.

However, early development decisions highlighted the risks of incomplete requirement understanding. A QR-code-based tracking system was initially implemented and functioned correctly from a technical perspective. Despite this, stakeholder feedback revealed that dedicated GPS tracking devices were already in use. As a result, the feature was removed.

This experience emphasised the importance of validating assumptions early in the development process. While the QR system demonstrated technical capability, it ultimately did not provide value to the end users. This reinforced the principle that successful software is defined not by its complexity, but by its relevance to user needs.

7.3 Scope Management and Trade-Offs

A key challenge throughout the project was managing scope within limited time constraints. Stakeholders expressed a requirement for a GDPR-compliant messaging system, as existing tools such as WhatsApp were not suitable due to compliance concerns.

While this feature would have significantly enhanced the system, it was ultimately deferred. Implementing a secure messaging platform would require additional considerations, including data encryption, secure storage, and compliance with data protection regulations. Attempting to include this feature risked compromising the quality and completeness of core system functionality.

This decision reflects a critical trade-off between feature completeness and system reliability. Prioritising core features such as dispatch, tracking, and proof-of-delivery ensured that the system delivered immediate value, even if some desirable functionality was postponed.

7.4 Technical Challenges

Several technical challenges were encountered during implementation, particularly in relation to system integration and reliability.

One notable issue involved the delivery of email receipts. Initial attempts resulted in emails being flagged as spam, reducing the effectiveness of the proof-of-delivery feature. This problem required investigation into email authentication mechanisms, leading to the implementation of domain verification, SPF, and DKIM configuration within AWS SES.

This experience highlighted the importance of considering real-world deployment factors beyond core functionality. While the feature was technically complete, its practical effectiveness depended on external systems and proper configuration.

Another challenge involved the implementation of real-time tracking. While WebSocket-based solutions were considered, the decision was made to use HTTP polling due to its simplicity and reliability within the project scope. This trade-off prioritised ease of implementation and stability over optimal efficiency.

7.5 Team Collaboration

The project was developed collaboratively by three team members, with responsibilities distributed across frontend, backend, and infrastructure components. Weekly meetings conducted via Discord provided a structured approach to planning and progress tracking.

This approach proved effective in maintaining consistent development progress. However, as with many team-based projects, challenges arose in coordinating work across different areas of the system. Ensuring compatibility between frontend and backend components required ongoing communication and integration testing.

The use of version control through GitHub played a crucial role in managing contributions and tracking changes. It also provided transparency in workload distribution and individual contributions.

7.6 Iterative Development and Learning

The iterative development approach adopted in this project allowed for continuous improvement based on feedback and testing. Each development cycle introduced new features, which were then refined based on internal testing and stakeholder input.

This process reinforced the value of incremental development, particularly in projects involving real users. Rather than attempting to design a complete system from the outset, the team was able to adapt to changing requirements and improve the system over time.

A key lesson learned was the importance of balancing technical ambition with practical feasibility. While it is tempting to implement advanced features, prioritising core functionality ensures that the system remains stable and usable.

7.7 Personal Reflection

From a personal perspective, this project provided valuable experience in full-stack development, cloud infrastructure, and stakeholder engagement. Working across multiple layers of the system, from frontend user interfaces to backend APIs and deployment infrastructure, highlighted the complexity of modern software systems.

The project also emphasised the importance of communication, both within the development team and with external stakeholders. Understanding user requirements, responding to feedback, and adapting to changing expectations are essential skills in real-world software engineering.

7.8 Summary

This reflection has highlighted the key lessons learned throughout the project, including the importance of stakeholder alignment, effective scope management, and consideration of real-world deployment challenges.

While the system successfully meets its core objectives, the development process provided valuable insights into the complexities of building practical, user-focused software. These lessons will inform future projects and contribute to continued development as a software engineer.

Chapter 8

Conclusion

8.1 Introduction

This chapter concludes the dissertation by summarising the outcomes of the Blood Bike West application, evaluating the extent to which the project objectives were achieved, and outlining potential areas for future development. It also reflects on the overall impact of the system and its relevance to real-world applications.

8.2 Achievement of Objectives

The primary objective of this project was to design and develop a web-based system to support the operational needs of Blood Bike West. This included implementing a job dispatch system, real-time tracking, authentication mechanisms, and proof-of-delivery functionality.

The results presented in the Evaluation chapter demonstrate that these objectives were largely achieved. The system successfully enables dispatchers to create and manage delivery jobs, while riders can view, accept, and complete tasks through a structured workflow.

The implementation of authentication and role-based access control ensures that users can only access functionality relevant to their role, improving both security and usability. Additionally, the proof-of-delivery system provides verifiable confirmation of completed deliveries, enhancing accountability within the organisation.

While real-time tracking was implemented using HTTP polling, this approach introduces minor delays compared to more advanced techniques such as WebSockets. As a result, this objective can be considered partially achieved, with opportunities for further refinement.

8.3 System Evaluation Summary

The evaluation of the system highlights several key strengths. The use of a serverless architecture enables scalability and reduces infrastructure management requirements. Performance testing demonstrated that the system is capable of handling concurrent users efficiently, particularly after optimisation efforts were applied.

One of the most significant findings was the identification and resolution of a performance bottleneck caused by incorrect environment configuration. Addressing this issue resulted in a substantial improvement in response times, demonstrating the importance of testing and optimisation in system development.

Usability testing indicated that the system is intuitive and accessible, particularly for mobile users. This is critical given the operational context of riders who rely on mobile devices in real-world scenarios.

8.4 Impact of Stakeholder Engagement

A key strength of this project was the direct involvement of Blood Bike West throughout the development process. This ensured that the system was aligned with real operational requirements rather than purely theoretical assumptions.

Stakeholder feedback played a crucial role in shaping the system. For example, the removal of the QR tracking feature and the introduction of proof-of-delivery functionality were direct results of stakeholder input. These changes improved the relevance and practicality of the final system.

This highlights the importance of user-centred design in software engineering, particularly when developing systems intended for real-world use.

8.5 Limitations of the System

Despite the success of the project, several limitations were identified.

Firstly, the system was not deployed in a full production environment, meaning that large-scale performance and reliability have not been fully validated. While testing was conducted under simulated conditions, real-world deployment may introduce additional challenges.

Secondly, the decision to use HTTP polling for real-time tracking introduces latency compared to more advanced solutions. While sufficient for current requirements, this approach may not scale effectively in larger systems.

Finally, certain features, such as a GDPR-compliant messaging system, were not implemented due to time constraints. While identified as valuable, these features were prioritised lower in favour of core system functionality.

8.6 Future Work

There are several opportunities for future development and improvement of the system.

One key area is the implementation of a real-time communication system using WebSockets, which would reduce latency and improve tracking accuracy.

Another area for development is the introduction of a GDPR-compliant messaging system. This would allow secure communication between users within the application, addressing a key stakeholder requirement.

Further improvements could include integration with existing GPS hardware, enhanced analytics and reporting features, and deployment in a production environment for real-world testing.

Additionally, expanding the system to support other volunteer-based organisations could increase its impact and applicability.

8.7 Final Reflection

This project demonstrates the successful application of modern web technologies and cloud infrastructure to solve a real-world problem. The system developed provides a functional and scalable solution for Blood Bike West, addressing key operational challenges.

The development process also provided valuable insights into software engineering practices, including stakeholder engagement, iterative development, and system optimisation.

Overall, the project highlights the importance of balancing technical capability with practical requirements, ensuring that systems are not only functional but also relevant and usable.

8.8 Summary

In conclusion, the Blood Bike West application meets its primary objectives and provides a strong foundation for future development. While limitations exist, the system demonstrates the potential for technology to improve the efficiency and effectiveness of volunteer-based medical logistics services.

Appendix

Project Resources

The full source code for this project is available at:
`https://github.com/AdamGallagher339/Codename-Blood`

A screencast demonstrating the system is available at:
`https://www.youtube.com/watch?v=LpsoNjBeT9w`

Bibliography

[1] Roy Fielding. Architectural styles and the design of network-based software architectures. *Doctoral Dissertation, University of California, Irvine*, 2000.

[2] Andreas Biørn-Hansen, Tim A. Majchrzak, and Tor-Morten Grønli. Progressive web apps: The future of web development? *Proceedings of the International Conference on Web Engineering*, 2017.

[3] Eiichi Taniguchi and Russell G. Thompson. *City Logistics: Network Modelling and Intelligent Transport Systems*. Pergamon, 2001.

[4] V. Pimentel and B. Nickerson. Performance comparison of websocket and http. *IEEE Internet Computing*, 2012.

[5] Ioana Baldini, Paul Castro, and Kerry Chang. Serverless computing: Current trends and open problems. *Research Advances in Cloud Computing*, 2017.

[6] Michael Jones, John Bradley, and Nat Sakimura. Json web token (jwt). RFC 7519, 2015.